



Security Audit

Black Tie (Bridge)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Centralisation	21
Conclusion	22
Our Methodology	23
Disclaimers	25
About Hashlock	26

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Black Tie Asset Hub team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

BT Asset Hub is an innovative digital platform developed by Black Tie Holdings Pty Ltd, designed to revolutionize traditional finance through asset tokenization. It enables asset managers to tokenize various asset classes—including real estate, mixed asset funds, lending and development funds, and businesses—by deploying smart contracts that convert physical assets into digital tokens. These tokenized assets are then listed on an integrated marketplace, where retail, wholesale, and institutional investors can access diverse investment opportunities using multiple payment methods, such as B4RC2, AUDD, USDT, and fiat currency. With its focus on compliance, security, and automated KYC/KYB processes, BT Asset Hub aims to democratize global investments, offering a user-friendly interface that empowers investors to seize control of their financial futures through blockchain technology.

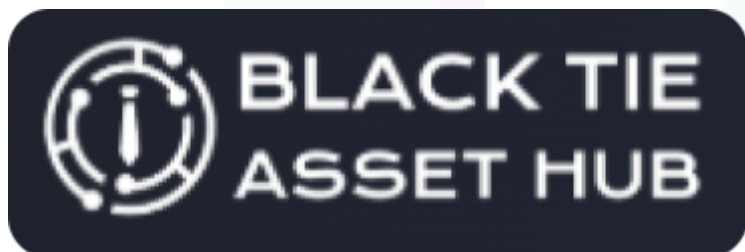
Project Name: Black Tie Asset Hub

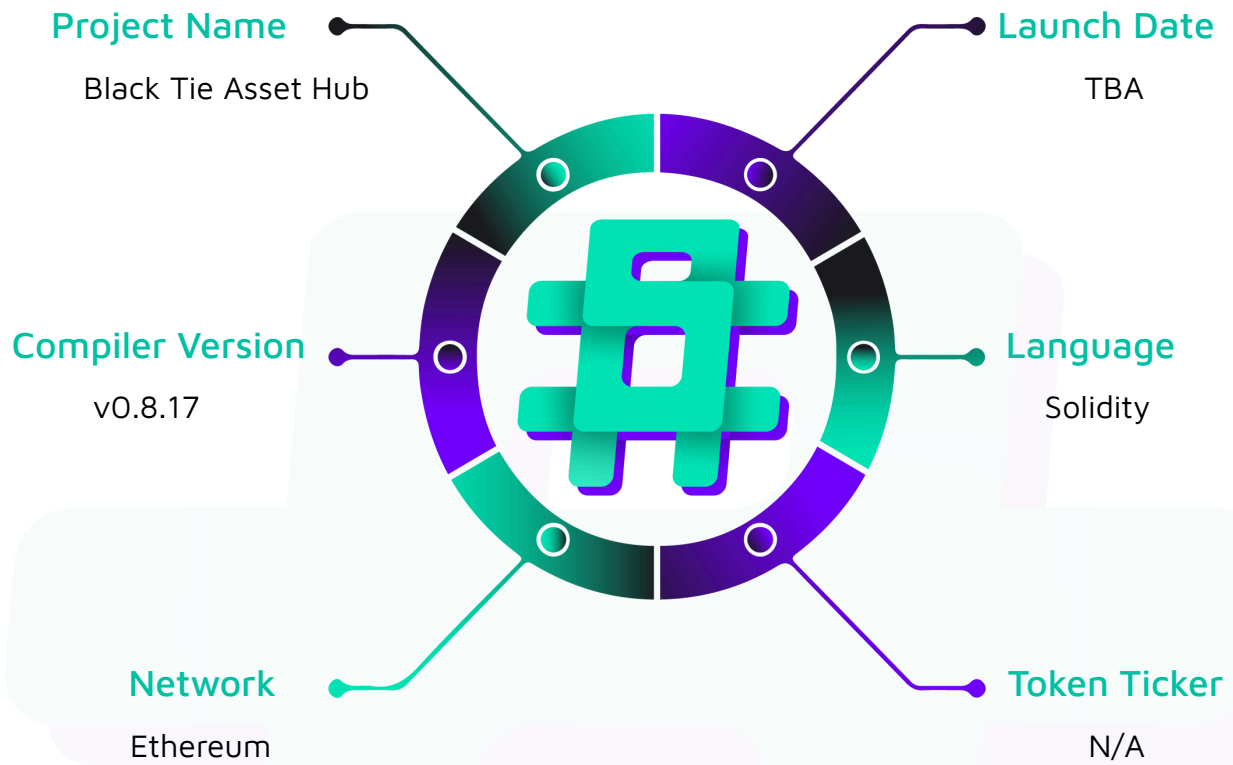
Project Type: Defi, Token, dApp, Vault, etc.

Compiler Version: v0.8.17

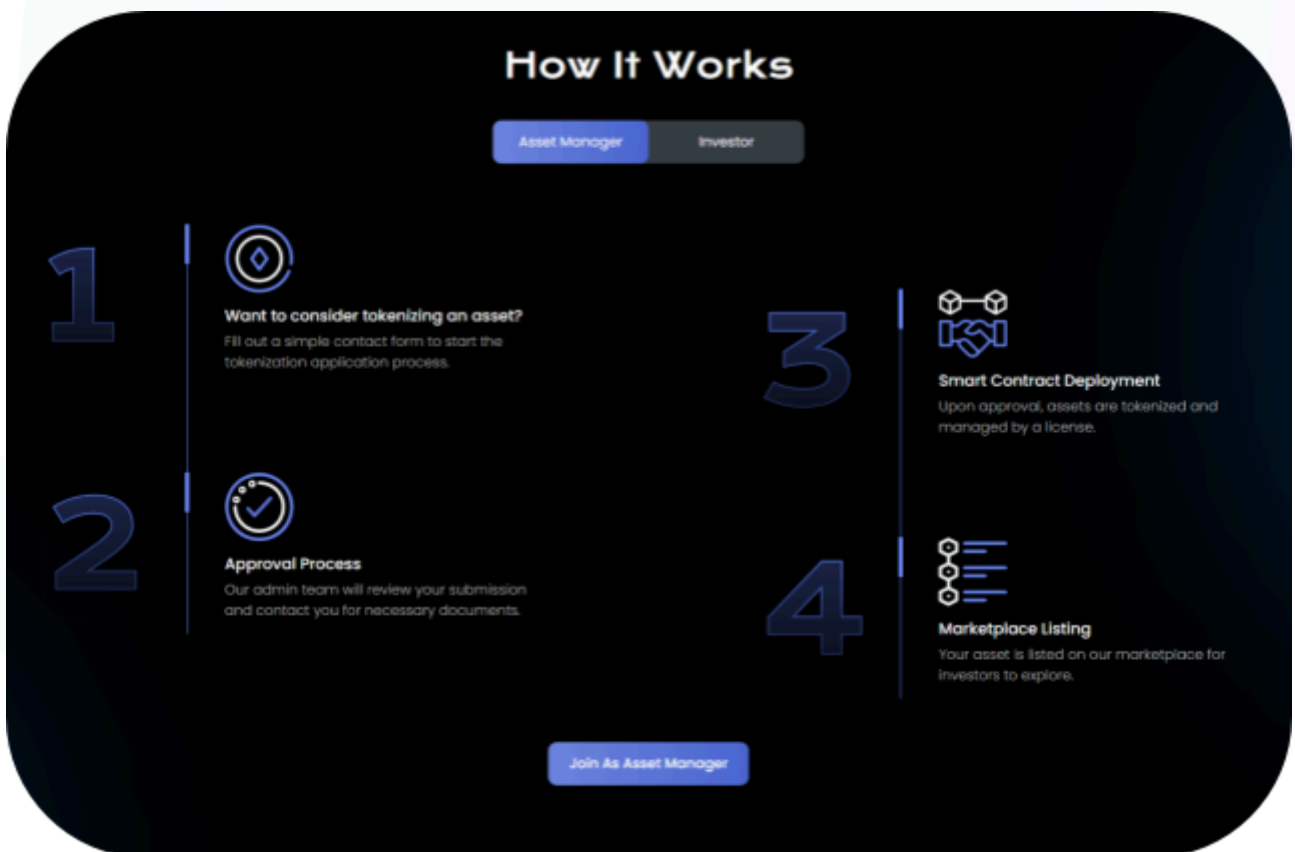
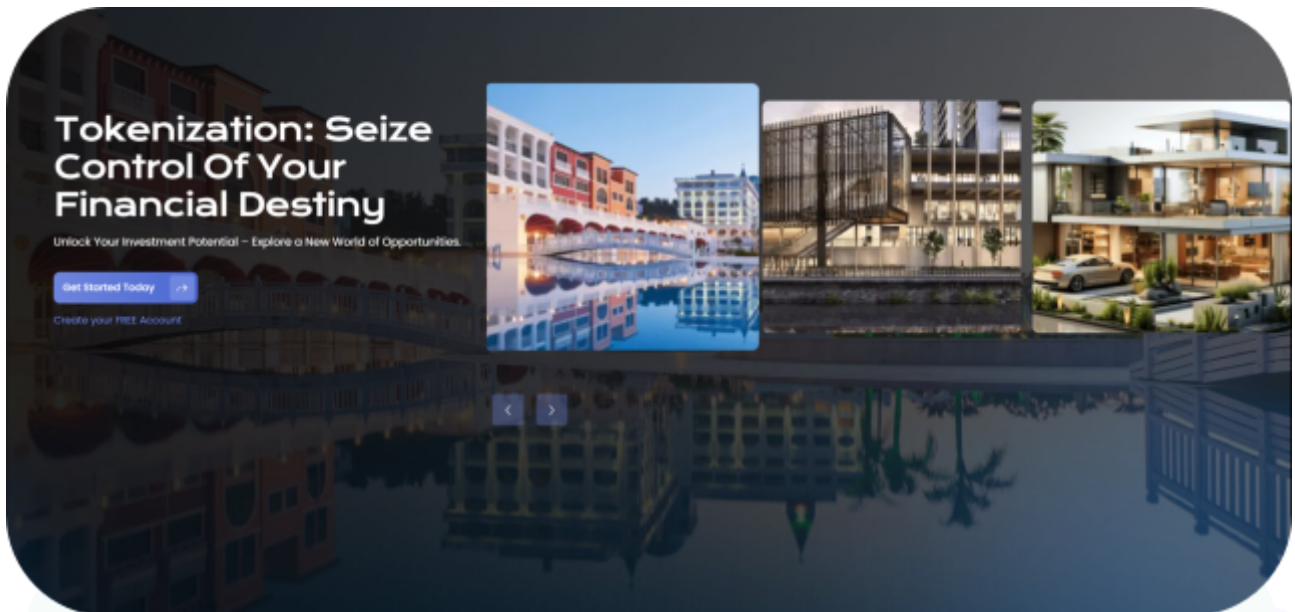
Website: <https://btassethub.digital/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Black Tie Asset Hub project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Black Tie Asset Hub Protocol Smart Contracts
Platform	Ethereum / Solidity
Audit Date	May, 2025
Contract 1	BridgeTieBridge.sol
Contract 1 MD5 Hash	0334448790a02176664f7718b3a3894b
Audited GitHub Commit Hash	d5ddfe5c36d7acae58643b1d8e1fed22d4d51ac9
Fix Review GitHub Commit Hash	09e30bcb88ec2d4c960d1403d92e88aca1738a8f

Note: Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit. For the "Fix Review GitHub Commit Hash", Hashlock solely verified the status of the identified findings and did not conduct a comprehensive review to assess any additional code implementations.

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 High severity vulnerabilities

6 Medium severity vulnerabilities

1 Low severity vulnerabilities

1 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
BlackTieBridge.sol - Allows users to: - Deposit tokens to bridge to other chain - Claim tokens on the destination chain by providing valid signatures - Allows admins to: - Update the bridge fees - Ability to change ownership (Ownable2Step)	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Black Tie Asset Hub project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Black Tie Asset Hub project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] BlackTieBridge#deposit - Strict Equality Check for Expected Amount leads to reverts

Description

The contract uses a strict equality check (==) to compare the expected amount provided by the user with the calculated amount on the Ethereum network – Due to potential rounding errors or precision differences in calculations, this strict check can cause legitimate transactions to fail.

Impact

Legitimate transactions might be rejected due to minor discrepancies between the calculated values and expected values provided by the user, leading to a poor user experience and reduced usability of the bridge.

Recommendation

Implement a tolerance range for the comparison instead of requiring exact matches. Or simply use <= instead of ==.

Status

Resolved

Medium

[M-01] BlackTieBridge#deposit - Users fail to withdraw their tokens on the corresponding Chain if they are using Account Abstraction Wallets.

Description

The Bridge assumes that users will have the same address on both chains by using `msg.sender` directly in the event emission and in the signature verification process. This fails to account for account abstraction wallets, which might have different addresses on different chains.

Impact

Users with account abstraction wallets, such as Safe Wallets, will face difficulties using the bridge, as the system assumes address equivalence across chains, limiting accessibility.

Recommendation

Allow specifying a recipient address on the destination chain during deposit. This would require modifying the deposit function and event structure.

```
function deposit(  
    uint256 _amount,  
    uint256 _expectedAmountOut,  
    address _recipientOnDestination  
) external nonReentrant {  
    // ... existing code ...  
    emit TokensDeposit(  
        msg.sender,  
        _recipientOnDestination, // Add recipient parameter  
        address(b4rc),
```

```

        _amount,

        _expectedAmountToGetE18,

        block.timestamp

    );
}

```

Status

Resolved

[M-02] BlackTieBridge#initialize - Missing Signer Validation During Initialization

Description

The contract initializes the list of signers without validating that they are valid addresses, Including `address(0)` in the signers list could break signature verification functionality – as users can provide fake signatures and can bypass the verification – and this particularly concerning since there appears to be no mechanism to update signers after initialization (CHECK M-05).

Impact

If an invalid address(especially `address(0)`) or duplicate address is included in the signers list, signature verification can be bypassed or will fail for all transactions, potentially rendering the bridge completely non-functional with no recovery path.

Recommendation

Implement validation to ensure no signer is `address(0)` and no duplicate exists during initialization. Additionally, consider adding functionality to update the signer list in case of key rotation or compromise of signers.

Status

Resolved

[M-03] BlackTieBridge#initialize - Missing Fee Validation leads to complete theft of funds as Fees or reverts

Description

The contract sets the fee percentage without validating that it is within a reasonable range. There's no check to ensure the fee doesn't exceed 100% (1000 basis points).

Impact

If fees are set to 100%, users lose all of their funds when bridging tokens, and if it is set more than 100%, it reverts due to underflows, effectively making the bridge unusable.

Recommendation

Add validation to ensure the fees are within a reasonable range in the initialize function, or directly pass the value to updateFees function.

Status

Resolved

[M-04] BlackTieBridge#initialize - Lack of Chain ID Validation

Description

The contract initialises `destinationChainId`, `ethChainId`, and `polChainId` without any validation checks. There's no verification that these values are non-zero or represent valid network IDs.

Impact

Invalid Chain IDs could cause bridge transactions to be routed incorrectly or rejected. If chain IDs are set to zero or invalid values, this could break core bridge functionality.

Recommendation

Add validation to ensure all chain IDs are non-zero and represent valid networks. It is recommended to hardcode these values and make them immutable as these are the values that won't be changed in the future.

Status

Resolved

[M-05] BlackTieBridge - Lack of Configuration Update Functions and Lack of Validations of Inputs

Description

The contract does not validate the inputs, such as `receiverAddress` parameter during initialization. This address is used to receive fee payments, so setting such values to `address(0)` or an incorrect address leads to loss of fees. Additionally, the contract does not provide functions to update critical configuration parameters such as token address, price aggregator, and receiver address after initialization.

Impact

If any of these parameters need to be changed (e.g due to a token upgrade or price feed change or for any other reason), the contract would need to be redeployed, potentially causing disruption to users and loss of state.

Recommendation

Implement Setter functions with appropriate access controls to allow updating critical parameters. Such as `updateB4rcToken`, `updatePriceAggregator`, `updateReceiverAddress` etc.

Status

Resolved

[M-06] BlackTieBridge#claim&calculateTokens - Use of Deprecated Chainlink's `latestAnswer()` function

Description

In the functions `claim` and `calculateTokens`, the deprecated `latestAnswer` function is used to retrieve prices from Chainlink:

```
function calculateTokens(
```



```

uint256 _amount,
uint256 _chainId
) public view returns (uint256 _expectedAmountToGetInE18) {
    require(chainId() == ethChainId, "Invalid chainId");
    if (_chainId == ethChainId) {
        _expectedAmountToGetInE18 =
            (_amount * (10 ** priceAggregator.decimals()) * (10 ** 18)) /
            (uint256(priceAggregator.latestAnswer()) *
            (10 ** b4rc.decimals()));
    } else {
        _amount = _amount * (10 ** 18);
        _expectedAmountToGetInE18 =
            (_amount * uint256(priceAggregator.latestAnswer())) /
            ((10 ** priceAggregator.decimals()));
    }
}

function claim(){
    if (chainId() == ethChainId) { // NOTE : formula = amount * <price> * 10^6
        // Oracle Price feed only available on ETH
        _expectedAmountToGet =
            (_amount *
            uint256(priceAggregator.latestAnswer()) *
            10 ** b4rc.decimals()) /
            ((10 ** priceAggregator.decimals()) * 10 ** 18);
    } else {
        _expectedAmountToGet = (_amount * 10 ** b4rc.decimals()) / 10 ** 18;
    }
}
}

```

According to Chainlink's documentation ([API Reference](#)), the `latestAnswer` function is deprecated. This function does not throw an error if no answer has been reached, but instead returns 0, possibly causing an incorrect price to be fed to the different price feeds or even a Denial of Service by a division by zero.

Recommendation

Consider implementing the [latestRoundData\(\)](#) method and improving logic to take advantage of its improved features.



Status

Acknowledged

Low

[L-01] BlackTieBridge#checkSignatures - Strict Signer Order Requirement sometimes leads to reverts.

Description

The signature verification process requires signers to sign in the exact order defined in the `_signers` array – Though this is provided by the Off-chain component – this strict ordering requirement may cause unnecessary complexity in real-world operations when users directly interact at the smart-contract level via a manual process.

If signers provide signatures in a different order than the one defined in the contract, the transaction will be rejected even if all required signers have signed. This could lead to reverts in transaction processing.

NOTE: This can happen as the off-chain component may have the chance to provide valid signatures in any order.

Recommendation

Consider implementing a more flexible signature verification that allows signatures to be submitted in any order, as long as they come from authorized signers.

Status

Acknowledged

QA

[Q-01] BlackTieBridge# - Redundant Chain Id Check

Description

The `calculateTokens` function has redundant chain ID validation. As per the code, the `calculateTokens` can only be called on the "Ethereum" chain. The function is only used in the `deposit` function when users make deposits on Ethereum as their source chain. As per the if logic at L-109, the `deposit` calls `calculateTokens` only when the source chain is `ETHChainId`. And again, inside the `calculateTokens`, the function makes the check again, making it redundant.

While this doesn't present a security vulnerability, it is inefficient and could cause confusion during code maintenance.

Recommendation

Remove the redundant check in the `calculateTokens` function, as it is being implemented inside the `deposit` if the source chain is Ethereum.

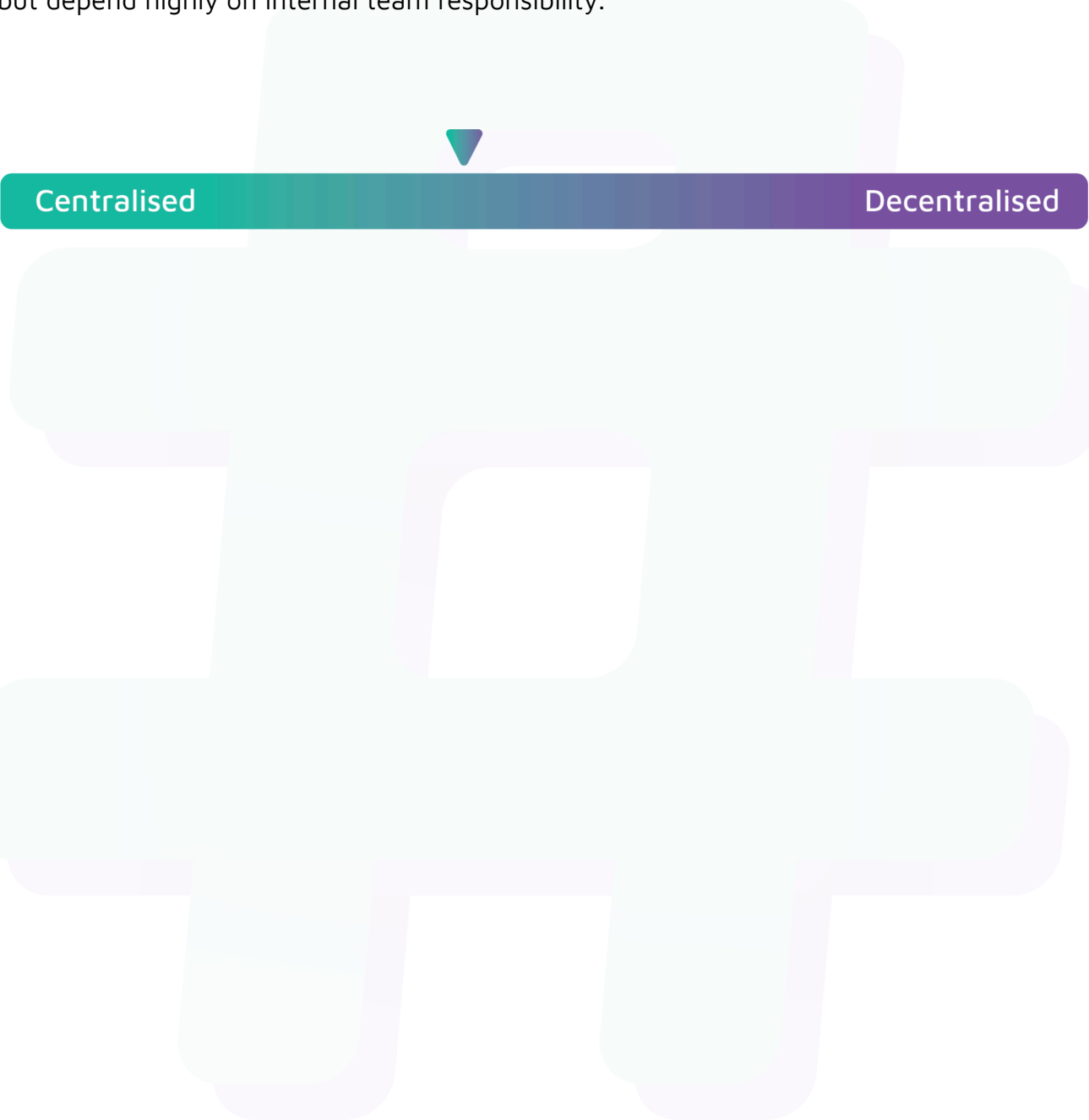
Status

Resolved

Centralisation

The Black Tie Asset Hub project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Black Tie Asset Hub project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.